

**PERANCANGAN UI *ONLINE CONTROLLER*
UNTUK *MULTI-DEVICE***

TUGAS AKHIR

Karya Ilmiah sebagai salah satu syarat untuk menyelesaikan jenjang Strata Satu
(S-1) di Departemen Teknik Elektro, Fakultas Teknik, Universitas Andalas

Oleh:

Zikri Nur Rahman

NIM. 2010952017

Pembimbing:

Darmawan, M.Sc

NIP. 197708162005011002



**Program Studi Sarjana
Teknik Elektro Fakultas Teknik
Universitas Andalas
2025**

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi yang pesat, sejalan dengan hadirnya revolusi industri 4.0, telah mendorong inovasi masif di berbagai sektor kehidupan. Era ini ditandai dengan fokus pada otomatisasi dan kolaborasi antara sistem siber dan fisik, yang bertujuan untuk meningkatkan efisiensi dan produktivitas. Salah satu teknologi pilar yang menjadi penggerak utama dalam transformasi ini adalah *Internet of Things* (IoT), sebuah konsep yang memperluas manfaat konektivitas internet ke berbagai perangkat fisik secara terus-menerus.[1]

Dewasa ini, perangkat yang terhubung ke internet atau Internet of Things (IoT) telah berkembang semakin pesat hingga menjadi bagian integral dari kehidupan sehari-hari. Barang-barang seperti lampu pintar, kamera keamanan, dan perangkat rumah tangga lainnya yang berbasis IoT kini hadir untuk memudahkan manusia dalam mengelola aktivitas sehari-hari dengan cara yang lebih efisien dan terintegrasi.[2]

Namun, kemajuan ini juga menghadirkan tantangan baru, terutama terkait pengendalian perangkat-perangkat tersebut. Banyak perangkat IoT memiliki kontroler atau pengendali masing-masing, yang sering kali membuat pengguna harus menggunakan beberapa aplikasi atau perangkat pengendali untuk mengelola berbagai perangkat yang berbeda. Situasi ini tidak hanya membingungkan tetapi juga dapat mengurangi efisiensi penggunaan teknologi IoT itu sendiri.[3]

Penelitian ini bertujuan untuk mengembangkan suatu sistem kontroler berbasis mobile yang mampu mengendalikan berbagai perangkat IoT secara terintegrasi. Dengan menggunakan antarmuka ini sebagai kontroler, pengguna dapat dengan mudah mengakses dan mengelola perangkat. Selain itu, pendekatan ini memungkinkan kontrol secara real-time sehingga meningkatkan efisiensi sekaligus kenyamanan pengguna.[4]

Penggunaan kontroler berbasis mobile juga memberikan fleksibilitas lebih tinggi karena pada dasarnya hampir semua orang sudah memiliki smartphone dan memungkinkan pengendalian perangkat dari berbagai lokasi selama terdapat koneksi internet. Dengan demikian, penelitian ini diharapkan dapat memberikan solusi praktis dan inovatif bagi tantangan pengendalian perangkat IoT secara efisien dan terintegrasi.[4]

Namun, implementasi sistem kontrol untuk banyak perangkat IoT tidak lepas dari tantangan teknis yang krusial, terutama terkait kualitas koneksi jaringan. Dalam sistem kendali real-time, keandalan dan kecepatan pengiriman data antara kontroler (aplikasi mobile) dan perangkat (aktuator IoT) adalah faktor penentu keberhasilan. Dua parameter kinerja jaringan yang paling berpengaruh di sini adalah throughput dan delay (latensi). Delay yang tinggi dapat menyebabkan

keterlambatan dalam pengiriman sinyal kontrol, sehingga aksi yang dilakukan oleh kontroler didasarkan pada informasi sensor yang sudah usang. Di sisi lain, throughput yang rendah dapat menghambat kemampuan sistem untuk mengelola banyak perangkat secara bersamaan, menyebabkan antrian data atau bahkan kehilangan paket informasi penting. Oleh karena itu, pengukuran dan optimisasi parameter kinerja jaringan ini menjadi fundamental untuk memastikan bahwa perintah kontrol dapat dikirimkan secara presisi dan stabil, sehingga sistem dapat berfungsi secara efektif dan responsif.[5]

Pada tahun 2019, telah dilakukan penelitian oleh Shameem Ahmad, dkk., dengan judul “Smart Home Automation And Security System Design Based On Iot Applications”. Penelitian tersebut berhasil merancang sebuah sistem otomasi rumah pintar yang komprehensif menggunakan Raspberry Pi sebagai kontroler pusat dan aplikasi mobile Android sebagai antarmuka pengguna. Sistem ini mampu mengendalikan beragam perangkat seperti lampu, kipas, dan AC, serta dilengkapi fitur keamanan dengan sensor gerak PIR dan pemantauan konsumsi daya. Namun, kendali yang diimplementasikan pada penelitian tersebut masih bersifat kendali langsung, seperti on/off dan pengaturan intensitas secara manual melalui slider. Penelitian tersebut belum menerapkan algoritma kendali closed-loop (umpan balik tertutup) yang diperlukan untuk perangkat yang membutuhkan pengaturan presisi dan stabilitas secara real-time. Selain itu, tidak ditemukan adanya analisis performa jaringan terkait parameter krusial seperti throughput dan delay, yang mana menjadi fokus utama pada penelitian yang akan dilakukan ini untuk menjamin keandalan sistem kendali secara nirkabel.[6]

Selanjutnya, penelitian yang relevan dilakukan oleh Rizky Hansza dan Subuh Isnur Haryudo pada tahun 2020 dengan judul “Rancang Bangun Kontrol Motor DC dengan PID menggunakan perintah suara dan Monitoring berbasis Internet Of Things (IOT)”. Penelitian ini berhasil menerapkan kontroler PID secara nirkabel untuk mengatur kecepatan sebuah motor DC. Inovasi utama pada penelitian ini adalah penggunaan perintah suara (voice recognition) sebagai setpoint input dan pemanfaatan platform Blynk untuk monitoring kondisi motor (arus, tegangan, dan kecepatan putar) secara real-time melalui internet. Meskipun telah berhasil menerapkan PID dalam kerangka IoT, sistem yang dibangun masih bersifat single-purpose, yaitu hanya dirancang untuk mengendalikan satu jenis perangkat saja (motor DC). Sistem tersebut belum dirancang sebagai sebuah platform terintegrasi yang mampu mengendalikan berbagai macam perangkat IoT berbeda dari satu antarmuka mobile yang sama. Selain itu, fokus penelitian ini adalah pada fungsionalitas kendali dan belum melakukan analisis performa jaringan untuk mengukur dampak throughput dan delay terhadap presisi serta stabilitas kendali, yang merupakan salah satu kebaruan utama dalam penelitian yang akan dilakukan ini.[7]

Berdasarkan tinjauan penelitian terkait, dapat disimpulkan bahwa penelitian sebelumnya cenderung berfokus pada salah satu aspek saja,

pengembangan antarmuka mobile untuk multi-perangkat tanpa algoritma kendali secara real-time atau implementasi perangkat kontroler untuk satu perangkat. Belum ada penelitian yang menggabungkan semuanya dalam satu sistem yang utuh.

Oleh karena itu, penelitian ini akan merancang sebuah sistem kontroler berbasis mobile yang mampu mengintegrasikan pengendalian banyak perangkat IoT secara terpusat. Kebaruan utama dari penelitian ini terletak pada analisis dan pengukuran performa sistem berdasarkan parameter throughput dan delay untuk memastikan kendali yang presisi dan stabil, sebuah aspek krusial yang belum diimplementasikan secara praktis pada sistem kendali mobile multi-perangkat sebelumnya. Dengan demikian, penelitian ini mengisi celah antara kemudahan penggunaan antarmuka mobile dengan keandalan sistem kendali presisi yang sensitif terhadap kualitas jaringan.

1.2 Rumusan Masalah

Rumusan masalah dari penelitian ini adalah sebagai berikut:

1. Bagaimana merancang sistem kontrol berbasis mobile yang dapat mengintegrasikan pengendalian banyak perangkat IoT yang bisa dikendalikan secara real-time?
2. Bagaimana mengukur dan mengoptimalkan performa sistem kontrol berbasis mobile berdasarkan parameter throughput dan delay untuk memastikan kendali yang presisi dan stabil?

1.3 Tujuan Penelitian

Tujuan dari penelitian ini adalah sebagai berikut:

1. Merancang sistem kontroler berbasis mobile yang mampu mengintegrasikan pengendalian banyak perangkat IoT yang bisa dikendalikan secara real-time.
2. Mengukur dan menganalisis kinerja sistem kontrol berdasarkan parameter throughput dan delay untuk memastikan efektivitas dan efisiensi kendali.

1.4 Manfaat Penelitian

Manfaat dari penelitian ini adalah sebagai berikut:

1. Menyediakan referensi ilmiah mengenai penerapan kontrol dalam sistem kontrol berbasis mobile untuk perangkat IoT.
2. Memberikan kontribusi terhadap pengembangan teknologi kontrol terpusat yang efisien dan terintegrasi.
3. Meningkatkan performa pengendalian sistem melalui optimasi throughput dan delay.

1.5 Batasan Masalah

Penelitian ini memiliki batasan masalah sebagai berikut:

1. Penelitian ini berfokus pada pengendali nirkabel.
2. Penelitian ini berfokus pada perangkat mobile.
3. Penelitian ini berfokus pada User Interface (UI) dan bukan User Experience (UX)

1.6 Sistematika Penulisan

BAB I PENDAHULUAN

Bab ini membahas mencakup pembahasan mengenai latar belakang dari penelitian, rumusan masalah, tujuan yang akan dicapai, batasan-batasan masalah, manfaat penelitian dan sistematika penulisan.

BAB II TINJAUAN PUSTAKA

Bab ini membahas mengenai landasan teori yang mendukung penyelesaian masalah pada penelitian ini.

BAB III METODOLOGI

Bab ini berisikan penjelasan mengenai metode yang mencakup diagram alir penelitian, prinsip kerja, bahan yang digunakan, perancangan jaringan dan teknik pengujian yang dilakukan.

BAB IV HASIL DAN ANALISA

Bab ini berisikan informasi hasil dan pembahasan dari penelitian tugas akhir ini.

BAB V KESIMPULAN DAN SARAN

Bab ini berisikan kesimpulan dari penelitian yang telah dilakukan dan saran untuk penelitian selanjutnya.

BAB II DASAR TEORI

2.1 Internet of Things (IoT)

2.1.1 Definisi dan Pertumbuhan

Internet of Things (IoT) didefinisikan sebagai sebuah paradigma di mana objek-objek di dunia fisik dilengkapi dengan sensor, aktuator, dan kemampuan komputasi serta komunikasi untuk terhubung ke internet, memungkinkan mereka untuk mengumpulkan dan bertukar data secara otonom. Konsep ini mentransformasi benda-benda statis menjadi entitas cerdas yang dapat berinteraksi dengan lingkungannya dan dengan sistem lain. Pertumbuhan IoT sangat pesat, didorong oleh miniaturisasi sensor, ketersediaan konektivitas nirkabel berbiaya rendah, dan kemajuan dalam komputasi awan (cloud computing). Statistik menunjukkan peningkatan jumlah perangkat terhubung dari 1,9 miliar pada tahun 2018 menjadi proyeksi 40 miliar pada tahun 2030, yang menandakan pergeseran fundamental menuju dunia yang hiper-terkoneksi.[8]

2.1.2 Arsitektur Sistem IoT

Untuk mengelola kompleksitasnya, sistem IoT umumnya diorganisir dalam sebuah arsitektur berlapis (layered architecture)[9]. Meskipun terdapat beberapa variasi model, arsitektur 4-lapis adalah yang paling umum diterima. Model 4-lapis terdiri dari :

1. Sensing Layer

Merupakan lapisan fisik yang berinteraksi langsung dengan lingkungan. Lapisan ini terdiri dari sensor untuk mengumpulkan data (misalnya, suhu, kelembaban, tekanan) dan aktuator untuk melakukan tindakan fisik (misalnya, membuka katup, menyalakan pemanas).

2. Network Layer

Bertanggung jawab untuk mentransmisikan data yang dikumpulkan oleh sensing layer ke sistem pemrosesan. Lapisan ini mencakup berbagai teknologi komunikasi nirkabel (Wi-Fi, Bluetooth, Seluler) dan protokol jaringan.

3. Data Processing Layer

Di lapisan ini, data mentah dari perangkat diolah, dianalisis, dan disimpan. Ini bisa terjadi di edge devices untuk respons cepat atau di cloud server untuk analisis data besar dan penyimpanan jangka panjang.

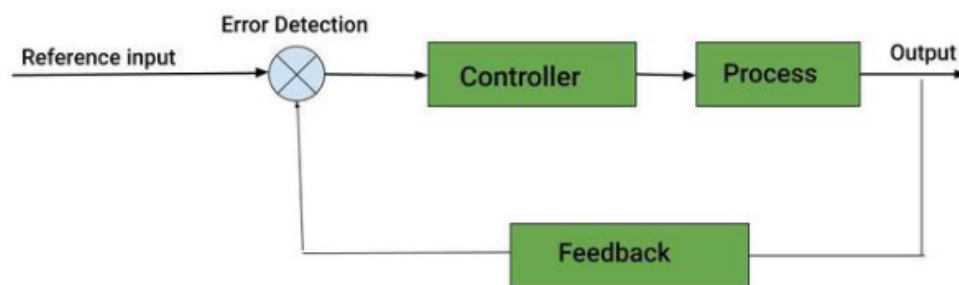
4. Application Layer

Merupakan lapisan teratas yang menyediakan antarmuka bagi pengguna akhir. Lapisan ini menerjemahkan data yang telah diproses menjadi informasi yang berguna dan dapat ditindaklanjuti, biasanya melalui aplikasi mobile, dasbor web, atau integrasi dengan sistem bisnis lainnya.

Sistem yang akan dirancang dalam penelitian ini mencakup semua lapisan tersebut, dengan aplikasi mobile berada di Application Layer, server Node.js di Data Processing Layer, protokol HTTP di Network Layer, dan perangkat keras di Sensing Layer.

2.2 Sistem Kontrol Closed Loop

Sistem kontrol closed-loop atau sistem kontrol umpan balik (feedback control) adalah sebuah sistem di mana sinyal keluaran (output) diukur dan dibandingkan dengan sinyal masukan referensi (input atau setpoint). Perbedaan antara keduanya, yang disebut sinyal error, kemudian digunakan oleh pengendali untuk menghasilkan sinyal kontrol yang akan memanipulasi proses agar keluaran sistem mendekati nilai yang diinginkan. Mekanisme umpan balik ini memungkinkan sistem untuk secara otomatis mengoreksi dirinya sendiri terhadap gangguan eksternal dan perubahan dalam dinamika proses, menjadikannya sangat andal dan banyak digunakan dalam aplikasi rekayasa.[10]



Gambar 2.1 Diagram Blok Sistem Kontrol Closed Loop

2.3 Arsitektur dan Komunikasi Data

2.3.1 Model Arsitektur Client-Server

Arsitektur client-server adalah model komputasi terdistribusi yang memisahkan peran antara penyedia layanan (server) dan peminta layanan (client). Dalam model ini, client menginisiasi permintaan ke server melalui jaringan. Server kemudian memproses permintaan tersebut, mengakses sumber daya yang diperlukan (seperti database), dan mengirimkan respons kembali ke client[11]. Arsitektur ini sangat scalable dan terpusat, memungkinkan banyak client untuk mengakses sumber daya yang sama secara bersamaan. Dalam konteks penelitian ini, arsitektur sistem akan mengikuti model tiga tingkat (three-tier):

1. Client tier, aplikasi mobile yang berjalan di perangkat pengguna.
2. Application/business logic tier, backend server berbasis Node.js yang menangani logika bisnis, manajemen perangkat, dan perutean permintaan.
3. Data/resource tier, perangkat-perangkat IoT fisik yang bertindak sebagai sumber daya yang dikontrol.

2.3.2 Protokol Komunikasi untuk IoT

Pemilihan protokol komunikasi sangat menentukan efisiensi, latensi, dan keandalan sistem IoT. Dua protokol yang paling dominan di lapisan aplikasi adalah HTTP dan MQTT.

1. HTTP (Hypertext Transfer Protocol)

HTTP merupakan protokol dasar dari World Wide Web. HTTP bekerja dengan model request-response, di mana client mengirimkan permintaan (misalnya, GET, POST) ke server, dan server merespons permintaan tersebut. Keunggulan utama HTTP adalah kesederhanaan, familiaritas bagi pengembang web, dan dukungan infrastruktur yang luas. Namun, untuk aplikasi IoT, HTTP memiliki kelemahan, yaitu bersifat stateless dan memiliki overhead header yang relatif besar untuk setiap transaksi. Hal ini membuatnya kurang efisien untuk komunikasi yang sering dan berukuran kecil, serta tidak mendukung komunikasi push dari server ke client secara native tanpa teknik seperti long polling[12].

2. MQTT (Message Queuing Telemetry Transport)

MQTT dirancang khusus untuk lingkungan IoT dengan keterbatasan bandwidth dan sumber daya. MQTT menggunakan model publish-subscribe (pub/sub) yang dimediasi oleh sebuah broker. Perangkat (publisher) mengirim pesan ke sebuah "topik" di broker, dan semua client yang "berlangganan" (subscribe) ke topik tersebut akan menerima pesan secara real-time. Model ini bersifat asynchronous, memiliki overhead protokol yang sangat kecil (minimal 2 byte), dan mendukung koneksi persisten. Fitur-fitur ini, ditambah dengan adanya Quality of Service (QoS) yang menjamin pengiriman pesan, membuat MQTT sangat unggul untuk komunikasi real-time, dua arah, dan efisien dalam penggunaan daya[12].

Meskipun MQTT secara teknis lebih unggul untuk banyak skenario IoT, penelitian ini memilih untuk menggunakan HTTP. Keputusan ini didasarkan pada pertimbangan untuk memprioritaskan kesederhanaan implementasi arsitektur RESTful API, yang sangat umum dan didukung dengan baik oleh framework seperti Express.js, serta untuk menghindari kompleksitas tambahan dalam mengelola MQTT broker terpisah. Evaluasi kinerja HTTP dalam skenario multi-perangkat ini justru akan memberikan data yang berharga mengenai batasan dan kelayakan penggunaannya.

Tabel 2.1: Perbandingan Protokol Komunikasi HTTP dan MQTT untuk Aplikasi IoT

Fitur	HTTP	MQTT
Model Arsitektur	Request-Response (Client-Server)	Publish-Subscribe (melalui Broker)

Overhead Protokol	Relatif tinggi, header tekstual	Sangat rendah, header biner minimal 2 byte
Manajemen Koneksi	Stateless, koneksi baru untuk setiap permintaan (dapat dioptimalkan dengan keep-alive)	Stateful, koneksi persisten untuk komunikasi berkelanjutan
Komunikasi Real-time	Kurang memerlukan dari client mendapatkan pembaruan	Sangat efisien, mendukung push pesan real-time dari broker ke subscriber
Quality of Service (QoS)	Tidak ada mekanisme QoS bawaan	Mendukung 3 level QoS (0, 1, 2) untuk menjamin pengiriman pesan
Kasus Penggunaan Ideal	Transfer data yang tidak sensitif waktu, interaksi dengan API web, perangkat dengan sumber daya cukup	Monitoring sensor real-time, kontrol perangkat jarak jauh, jaringan tidak stabil, perangkat dengan sumber daya terbatas

2.3.3 Format Data JSON (JavaScript Object Notation)

JSON adalah format pertukaran data berbasis teks yang ringan, mudah dibaca oleh manusia, dan mudah di-parsing oleh mesin. Strukturnya yang terdiri dari pasangan key-value dan dukungan untuk tipe data dasar (string, angka, boolean, array, objek) membuatnya sangat fleksibel. Dalam arsitektur yang diusulkan, JSON akan digunakan sebagai format payload untuk semua komunikasi API antara aplikasi mobile, server, dan perangkat IoT. Penggunaannya menyederhanakan proses serialisasi dan deserialisasi data di semua komponen sistem[13].

2.4 Teknologi Pengembangan Sistem

2.4.1 Pengembangan Aplikasi Mobile dengan React Native

React Native adalah sebuah framework open-source yang dikembangkan oleh Facebook untuk membangun aplikasi mobile untuk platform iOS dan Android menggunakan satu basis kode (codebase) yang sama, yaitu JavaScript dan React. Keunggulan utamanya adalah efisiensi pengembangan; pengembang tidak perlu menulis dan memelihara dua aplikasi terpisah. React Native mencapai

kinerja yang mendekati aplikasi native karena komponen UI yang ditulis dalam JavaScript akan di-render menjadi elemen UI native yang sesungguhnya, bukan dijalankan dalam sebuah WebView seperti pada pendekatan hibrida lainnya. Hal ini menjadikannya pilihan yang kuat untuk mengembangkan antarmuka kontrol IoT yang responsif dan memiliki performa tinggi[14].

2.4.2 Pengembangan Backend dengan Node.js dan Express.js

Node.js adalah sebuah lingkungan eksekusi (runtime environment) JavaScript sisi server yang dibangun di atas mesin V8 JavaScript milik Google Chrome. Fitur yang paling menonjol dari Node.js adalah model I/O-nya yang event-driven dan non-blocking. Berbeda dengan model multi-threaded tradisional di mana setiap koneksi yang masuk akan dilayani oleh satu thread terpisah, Node.js menggunakan satu thread utama (single-threaded event loop) untuk menangani semua koneksi secara asynchronous. Arsitektur ini membuatnya sangat efisien dalam menangani ribuan koneksi konkuren dengan konsumsi sumber daya (memori dan CPU) yang minimal. Karakteristik ini sangat cocok untuk aplikasi backend IoT, yang harus secara konstan berkomunikasi dengan banyak perangkat secara simultan[15].

Express.js adalah framework web yang minimalis dan fleksibel untuk Node.js. Express.js menyediakan serangkaian fitur yang kuat untuk aplikasi web dan mobile, termasuk sistem routing yang sederhana dan dukungan untuk middleware, yang sangat menyederhanakan proses pembuatan REST API untuk backend sistem ini[16].

2.4.3 Expo Go

Expo Go adalah aplikasi klien open-source untuk iOS dan Android yang memungkinkan pengembang untuk melihat dan menguji aplikasi React Native yang sedang dikembangkan secara real-time langsung di perangkat fisik tanpa harus melakukan kompilasi native melalui Xcode atau Android Studio. Expo Go menyederhanakan proses pengembangan awal secara signifikan. Pengembang cukup memindai kode QR dari terminal pengembangan (Expo CLI), dan aplikasi akan langsung dimuat di perangkat mereka melalui jaringan Wi-Fi. Selain itu, Expo Go sudah dilengkapi dengan berbagai API native yang siap pakai, seperti akses ke kamera, sensor, sistem file, dan notifikasi, sehingga mempercepat proses pembuatan prototipe dan pengembangan fitur tanpa perlu konfigurasi native yang rumit[17].

2.4.4 Database

Untuk basis data, penelitian ini menggunakan MongoDB, sebuah sistem basis data NoSQL yang berorientasi pada dokumen. Berbeda dengan basis data relasional (SQL) yang menggunakan tabel dengan skema yang kaku, MongoDB menyimpan data dalam format BSON (Binary JSON) yang fleksibel. Struktur ini

sangat cocok untuk aplikasi IoT di mana data yang diterima dari berbagai sensor dan perangkat bisa memiliki format dan atribut yang berbeda-beda, tanpa memerlukan perubahan skema database secara konstan.

Kemampuan MongoDB untuk menangani volume data yang besar dengan kecepatan tulis yang tinggi (high write throughput) dan skalabilitas horizontal melalui mekanisme sharding menjadikannya solusi yang andal untuk menampung data deret waktu (time-series) yang terus-menerus dihasilkan oleh banyak perangkat IoT secara simultan. Selain itu, karena data disimpan dalam format yang mirip dengan JSON, integrasinya dengan backend Node.js menjadi sangat mulus dan efisien, menyederhanakan proses pengembangan secara keseluruhan. Studi yang membandingkan performa basis data untuk aplikasi berbasis IoT menunjukkan bahwa basis data NoSQL seperti MongoDB menawarkan keunggulan signifikan dalam hal fleksibilitas skema dan skalabilitas untuk menangani data IoT yang heterogen dan bervolume besar dibandingkan dengan basis data SQL tradisional[18].

2.5 User Interface dan User Experience (UI/UX)

2.5.1 Prinsip-Prinsip Desain UI/UX

Desain user interface (UI) dan user experience (UX) adalah disiplin ilmu yang berfokus pada penciptaan produk digital yang tidak hanya fungsional tetapi juga mudah, efisien, dan menyenangkan untuk digunakan. Untuk merancang aplikasi kontrol multi-device yang efektif, beberapa prinsip desain fundamental harus diterapkan[19]:

1. User-Centered Design (Berpusat pada Pengguna)

Seluruh proses desain harus dimulai dari pemahaman mendalam tentang kebutuhan, tujuan, dan batasan pengguna. Ini melibatkan riset pengguna dan pengujian untuk memastikan desain akhir benar-benar intuitif bagi target penggunanya.

2. Konsistensi

Elemen desain (seperti tombol, ikon, tipografi) dan pola interaksi harus konsisten di seluruh aplikasi. Konsistensi mengurangi beban kognitif pengguna, membuat aplikasi lebih mudah dipelajari dan diprediksi.

3. Kesederhanaan (Simplicity)

Antarmuka harus dirancang seminimalis mungkin, dengan menghilangkan elemen-elemen yang tidak perlu. Desain yang sederhana dan bersih memungkinkan pengguna untuk fokus pada tugas utama mereka tanpa terdistraksi oleh kerumitan visual.

4. Visibilitas dan Feedback

Elemen dan informasi penting harus mudah terlihat dan diakses. Setiap tindakan yang dilakukan pengguna harus segera diikuti oleh umpan balik (feedback) yang jelas dari sistem. Dalam konteks sistem kontrol, feedback ini

sangat krusial; misalnya, setelah pengguna mengubah setpoint, aplikasi harus secara visual mengonfirmasi bahwa perintah telah berhasil dikirim dan diterima oleh perangkat.

5. Minimalisasi Kesalahan

Desain yang baik harus dapat mencegah pengguna melakukan kesalahan. Jika kesalahan tidak dapat dihindari, sistem harus memberikan pesan error yang jelas dan konstruktif, serta menyediakan cara mudah untuk membatalkan atau memperbaiki kesalahan tersebut. Dalam sistem kontrol, ini berarti mencegah pengguna memasukkan nilai parameter yang tidak valid atau berbahaya.

2.6 Quality of Service (QoS)

2.6.1 Throughput

Throughput adalah laju transfer data efektif yang berhasil dikirim melalui sebuah saluran komunikasi dalam satuan waktu tertentu, biasanya diukur dalam bit per detik (bps) atau megabit per detik (Mbps). Throughput berbeda dengan bandwidth, bandwidth adalah kapasitas teoritis maksimum dari saluran, sedangkan throughput adalah laju aktual yang dicapai, yang seringkali lebih rendah karena faktor-faktor seperti overhead protokol, kongesti jaringan, dan latensi. Dalam sistem ini, throughput yang memadai penting untuk memastikan data monitoring dari banyak perangkat dapat diterima oleh server tanpa penundaan yang signifikan[20].

$$Throughput = \frac{\text{Paket Data Diterima}}{\text{Lama pengamatan}} \quad (2.2)$$

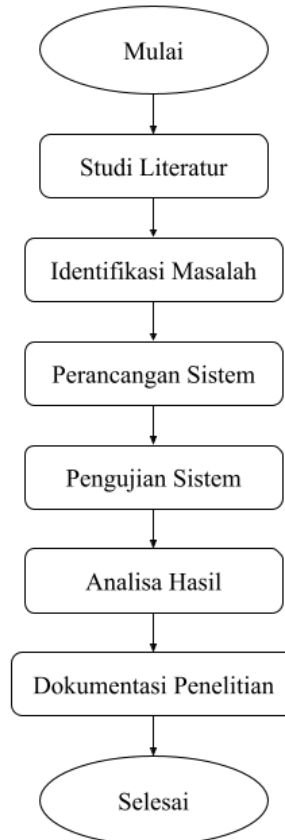
2.6.2 Delay (Latency)

Delay atau latency adalah waktu yang dibutuhkan oleh sebuah paket data untuk melakukan perjalanan dari titik sumber ke titik tujuan, biasanya diukur dalam milidetik (ms). Dalam sistem kontrol real-time, latensi adalah parameter yang sangat kritis. Latensi yang tinggi dalam pengiriman sinyal kontrol dari aplikasi ke perangkat, atau dalam penerimaan data status dari perangkat, dapat menyebabkan respons sistem yang lambat, ketidakstabilan, dan bahkan kegagalan kontrol. Total waktu tunda ini sering diukur sebagai Round-Trip Time (RTT), yaitu waktu dari pengiriman paket hingga penerimaan konfirmasi[20].

$$Delay = \frac{\text{Packet Length}}{\text{Link Bandwidth}} \quad (2.3)$$

BAB III METODOLOGI PENELITIAN

3.1 Tahap Penelitian



Gambar 3.1 Diagram Tahap Penelitian

1. Studi Literatur

Pada tahap ini penulis mengumpulkan referensi dan teori mengenai topik permasalahan yang akan diangkat. Sumber dan teori dapat berupa laporan penelitian terdahulu, jurnal, buku, tesis, dan lain sebagainya.

2. Identifikasi Masalah

Setelah mengumpulkan referensi dan teori, berdasarkan referensi tersebut didapatkan potensi permasalahan yang dapat diangkat menjadi judul penelitian.

3. Perancangan Sistem

Setelah dirumuskan masalah yang akan diangkat dalam penelitian dilakukan perancangan sistem yang menunjang sebagai jawaban dari masalah yang diangkat.

4. Pengujian Sistem

Sistem yang telah dibuat lalu diuji untuk mengetahui efektifitas dari sistem dan mendapatkan data, dan dilakukan penyesuaian ulang agar memenuhi spesifikasi yang dibutuhkan.

5. Analisa Hasil

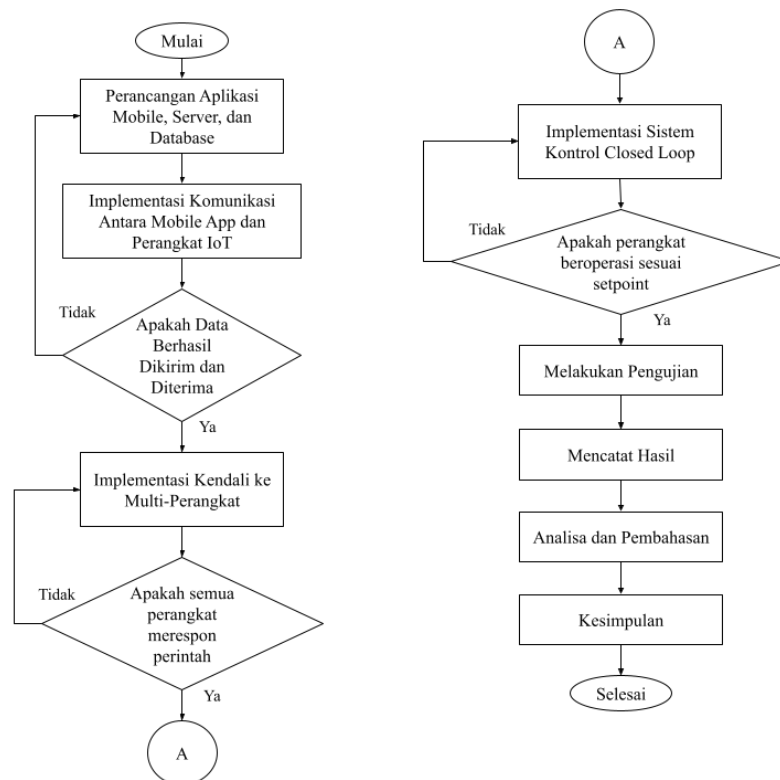
Sistem yang telah sesuai dengan spesifikasi akan menghasilkan data, data tersebut akan diolah dan dianalisa.

6. Dokumentasi Penelitian

Pada tahap ini merupakan pelaporan hasil penelitian yang telah dilakukan berupa dokumentasi tertulis.

3.2 Diagram Alir Penelitian

Untuk memberikan gambaran visual yang jelas mengenai alur kerja penelitian, sebuah diagram alir akan disajikan. Diagram ini akan mengilustrasikan urutan logis dari setiap tahapan penelitian.



Gambar 3.2 Diagram Alir Penelitian

3.3 Metodologi Komunikasi Data dan Arsitektur Sistem

Sistem yang dibuat akan dibangun di atas arsitektur client-server dua tingkat. Berikut adalah penjabarannya:

1. Mobile Client (React Native)

Komponen ini berjalan pada smartphone pengguna. Fungsinya adalah sebagai antarmuka utama (front-end) untuk interaksi pengguna. Aplikasi ini akan bertanggung jawab untuk menampilkan daftar perangkat yang terhubung, memvisualisasikan data telemetri (misalnya, grafik suhu real-time), dan mengirimkan perintah pengguna (misalnya, mengubah setpoint atau parameter PID) ke backend server melalui permintaan HTTP.

2. Backend Server (Node.js)

Bertindak sebagai otak dan perantara sistem (middleware). Server ini memiliki beberapa tanggung jawab utama:

- a) menyediakan REST API untuk berkomunikasi dengan mobile client.
- b) mengelola daftar perangkat yang terdaftar beserta statusnya.
- c) menerima, memvalidasi, dan merutekan perintah dari client ke perangkat IoT yang dituju.
- d) mengumpulkan data telemetri dari perangkat untuk kemudian diteruskan ke client.

Komunikasi antara kedua komponen ini akan diatur oleh sebuah REST API yang terdefinisi dengan baik. REST (Representational State Transfer) adalah gaya arsitektur yang menggunakan standar HTTP untuk operasi Create, Read, Update, Delete (CRUD) pada sumber daya (resources). Mengikuti praktik terbaik desain REST API, sumber daya akan diidentifikasi menggunakan noun (kata benda) dalam URI, dan operasi akan ditentukan oleh metode HTTP (GET, POST, PUT).

Tabel 3.1: Definisi Endpoint REST API untuk Kontrol Perangkat PID

Metode HTTP	URI Endpoint	Deskripsi	Request Body
GET	/devices	Mengambil daftar semua perangkat yang terdaftar di sistem, beserta status ringkasnya (ID, nama, status online/offline).	-
GET	/devices/{id}	Mengambil data status dan parameter terperinci dari satu perangkat spesifik yang diidentifikasi oleh {id}.	-
POST	/devices/{id}/command	Mengirimkan sebuah	{ "command":

	perintah ke perangkat spesifik. Perintah dapat berupa pembaruan <i>setpoint</i> atau parameter.	"update_params", "payload": { "setpoint": 50.0 } }
POST /data/{id}	Endpoint yang digunakan oleh perangkat IoT untuk mengirimkan data telemetri (<i>process variable</i> saat ini) ke <i>server</i> .	{ "process_variable": 45.5, "timestamp": "2024-10-27T10:00:00Z" }

3.4 Perancangan, Pengolahan, dan Penampilan Data

3.4.1 Perancangan Antarmuka Pengguna (UI)

Proses perancangan UI akan mengadopsi metodologi user-centered design untuk memastikan hasil akhir yang intuitif dan fungsional.

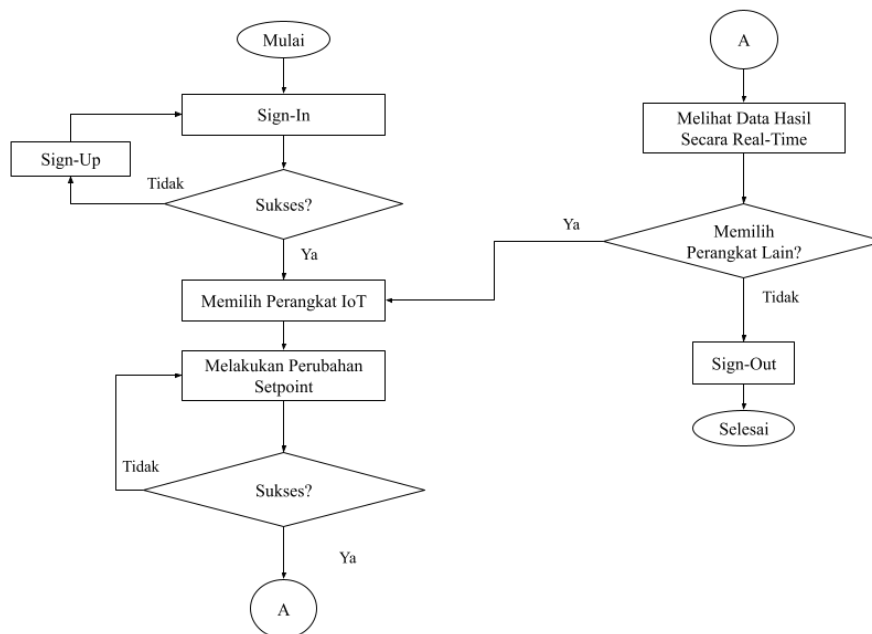
1. Wireframing

Tahap awal akan fokus pada pembuatan wireframe dengan tingkat ketelitian rendah (low-fidelity). Wireframe ini akan berfungsi sebagai cetak biru struktural, menentukan tata letak elemen-elemen utama, alur navigasi antar layar, dan hierarki informasi tanpa terganggu oleh detail visual seperti warna atau tipografi.

2. Mockup/Prototype

Setelah struktur disetujui, proses akan dilanjutkan dengan pembuatan mockup atau prototipe interaktif dengan tingkat ketelitian tinggi (high-fidelity). Pada tahap ini, prinsip-prinsip desain UI/UX yang telah dibahas di Bab 2 akan diterapkan. Desain visual, termasuk skema warna, tipografi, dan ikonografi, akan dikembangkan untuk menciptakan antarmuka yang estetis dan mudah digunakan. Antarmuka utama akan mencakup:

- a) Layar Daftar Perangkat
Menampilkan semua perangkat yang terhubung dengan status dasarnya.
- b) Layar Detail Perangkat
Menampilkan informasi mendalam untuk satu perangkat, termasuk grafik real-time yang memplot nilai setpoint versus process variable.
- c) Formulir Pendaftaran Perangkat
Layar terpisah yang memungkinkan pengguna untuk menambahkan perangkat baru.



Gambar 3.3 User Flow untuk pengguna

3.4.2 Logika Pengolahan Data di Sisi Server

Backend server akan menjadi komponen sentral dalam pengolahan data. Server akan mengelola state atau kondisi dari setiap perangkat IoT. Ini akan diimplementasikan dengan menggunakan sebuah database sederhana (mongodb) untuk menyimpan informasi penting setiap perangkat, seperti ID unik, MAC Address, alamat IP terakhir yang diketahui, dan timestamp komunikasi terakhir. Ketika sebuah perintah diterima dari aplikasi mobile, server akan mencari perangkat target berdasarkan ID-nya dan meneruskan perintah tersebut. Server juga akan menyediakan endpoint bagi perangkat untuk melaporkan data telemetrinya secara periodik.

3.4.3 Visualisasi Data pada Aplikasi Mobile

Untuk memberikan pemahaman yang cepat dan intuitif kepada pengguna mengenai kinerja sistem kontrol, data telemetri akan divisualisasikan secara grafis. Aplikasi mobile akan menggunakan library grafik (react-native-charts-wrapper) untuk menampilkan data deret waktu (time-series) dari process variable dan setpoint dalam bentuk diagram garis. Visualisasi ini memungkinkan pengguna untuk secara langsung mengamati respon sistem terhadap perubahan setpoint atau gangguan, seperti melihat adanya overshoot, settling time, dan steady-state error. Data seperti throughput dan juga delay akan ditampilkan di UI-nya.

3.5 Pengujian Sistem

3.5.1 Pengujian Fungsional

Pengujian ini dilakukan untuk memastikan bahwa setiap fitur dan fungsi sistem bekerja sesuai dengan spesifikasi yang telah dirancang. Pengujian akan menggunakan pendekatan black-box, di mana penguji berinteraksi dengan aplikasi tanpa perlu mengetahui detail implementasi internalnya. Skenario pengujian akan mencakup, namun tidak terbatas pada:

- a) Memverifikasi bahwa aplikasi berhasil menampilkan daftar perangkat yang aktif.
- b) Memastikan perubahan nilai setpoint pada aplikasi mobile untuk perangkat A benar-benar mengubah setpoint pada perangkat fisik A.
- c) Menguji validasi input untuk parameter (misalnya, menolak nilai negatif).
- d) Memverifikasi bahwa grafik data pada aplikasi diperbarui secara real-time ketika perangkat mengirim data baru.

3.5.2 Pengujian Kinerja Jaringan

Pengujian ini bertujuan untuk mengukur dampak skalabilitas (penambahan jumlah perangkat) terhadap parameter kinerja jaringan, yaitu throughput dan delay.

1. Lingkungan Pengujian

Pengujian akan dilakukan dalam lingkungan jaringan localhost yang terkontrol.

2. Prosedur:

1. Pengujian akan dilakukan dengan skenario beban yang berbeda, yaitu dengan jumlah perangkat aktif sebanyak 1, 3, 5, dan 10 unit.
2. Untuk setiap skenario, proses pengiriman 10 permintaan serentak akan diulang beberapa kali untuk mendapatkan data yang representatif.
3. Pengukuran delay (latency), waktu respons total, atau Round-Trip Time (RTT), akan dicatat untuk setiap pasangan permintaan-respons. RTT diukur dari saat permintaan dikirim oleh skrip pengujian hingga respons lengkap diterima dari server.
4. Pengukuran throughput, throughput akan dihitung sebagai jumlah total transaksi (permintaan + respons) yang berhasil diproses oleh server per satuan waktu (transaksi per detik) atau jumlah total data (byte) yang ditransfer per detik.

3.6 Metode Analisis Data

Data mentah dari pengujian kinerja jaringan (delay dan throughput) akan diolah dan dianalisis. Hasilnya akan disajikan dalam bentuk tabel ringkasan dan divisualisasikan menggunakan grafik, seperti grafik batang untuk membandingkan rata-rata delay, atau grafik garis untuk menunjukkan tren throughput seiring dengan penambahan beban perangkat. Analisis ini akan bertujuan untuk

mengidentifikasi apakah ada degradasi kinerja yang signifikan seiring dengan meningkatnya jumlah perangkat yang dikendalikan.

3.7 Jadwal Penelitian

No	Jenis Kegiatan	Bulan				
		Juli	Agustus	September	Oktober	November
1	Studi Literatur					
2	Perancangan Komunikasi Data					
3	Perancangan Sistem					
4	Pengujian dan pengambilan data QoS					
5	Analisa dan Penarikan Kesimpulan					
6	Penyusunan Laporan Tugas Akhir					

DAFTAR PUSTAKA

- [1] M. Jufri *dkk.*, “Dunia Teknologi Informasi & Revolusi Industri 4.0,” *jpb*, vol. 5, no. 2, Jun 2023, doi: 10.33884/jpb.v5i2.7343.
- [2] I. Ud Din *dkk.*, “The Internet of Things: A Review of Enabled Technologies and Future Challenges,” *IEEE Access*, vol. 7, hlm. 7606–7640, 2019, doi: 10.1109/ACCESS.2018.2886601.
- [3] S. S. Albouq, A. A. A. Sen, N. Almashf, M. Yamin, A. Alshanjiti, dan N. M. Bahbough, “A Survey of Interoperability Challenges and Solutions for Dealing With Them in IoT Environment,” *IEEE Access*, vol. 10, hlm. 36416–36428, 2022, doi: 10.1109/ACCESS.2022.3162219.
- [4] A. R. Al Tahtawi, E. A. Andika, dan W. N. Harjanto, “Desain Awal Pengembangan Sistem Kontrol Irigasi Otomatis Berbasis Node Nirkabel dan Internet-of-Things,” *Jurnal Otomasi Kontrol dan Instrumentasi*, vol. 10, no. 2, hlm. 485865, 2018, doi: 10.5614/joki.2018.10.2.5.
- [5] T. George dan V. Ganesan, “Optimal tuning of PID controller in time delay system: a review on various optimization techniques,” *Chemical Product and Process Modeling*, vol. 17, no. 1, hlm. 1–28, Feb 2022, doi: 10.1515/cppm-2020-2001.
- [6] S. Ahmad *dkk.*, “SMART HOME AUTOMATION AND SECURITY SYSTEM DESIGN BASED ON IOT APPLICATIONS,” *ASEAN Engineering Journal*, vol. 9, no. 2, hlm. 57–71, Des 2019, doi: 10.11113/aej.v9i15513.
- [7] R. Hansza dan S. I. Haryudo, “RANCANG BANGUN KONTROL MOTOR DC DENGAN PID MENGGUNAKAN PERINTAH SUARA DAN MONITORING BERBASIS INTERNET OF THINGS (IOT),” *JURNAL TEKNIK ELEKTRO*, vol. 9, no. 2, Feb 2020, doi: 10.26740/jte.v9n2.p%25p.
- [8] J. Shehu Yalli, M. Hilmi Hasan, dan A. Abubakar Badawi, “Internet of Things (IoT): Origins, Embedded Technologies, Smart Applications, and Its Growth in the Last Decade,” *IEEE Access*, vol. 12, hlm. 91357–91382, 2024, doi: 10.1109/ACCESS.2024.3418995.
- [9] “IoT Architecture: Six Levels, Core Components and Use Cases.” Diakses: 23 September 2025. [Daring]. Tersedia pada: <https://www.itransition.com/iot/architecture>
- [10] A. Sari dan F. Jusmi, “PERANCANGAN SISTEM KONTROL PID DENGAN APLIKASI SCILAB,” *Applied Physics of Cokroaminoto Palopo*, vol. 2, no. 1, hlm. 31–41, Des 2021, doi: 10.30605/apcp.v2i1.86.
- [11] School of Computing Universiti Utara Malaysia Kedah, Malaysia dan H. S. Oluwatosin, “Client-Server Model,” *IOSRJCE*, vol. 16, no. 1, hlm. 57–71, 2014, doi: 10.9790/0661-16195771.
- [12] T. Yokotani dan Y. Sasaki, “Comparison with HTTP and MQTT on required network resources for IoT,” dalam *2016 International Conference on Control, Electronics, Renewable Energy and Communications (ICCEREC)*, Sep 2016, hlm. 1–6. doi: 10.1109/ICCEREC.2016.7814989.
- [13] P. Bourhis, J. L. Reutter, F. Suárez, dan D. Vrgoč, “JSON: Data model, Query languages and Schema specification,” dalam *Proceedings of the 36th*

- ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, dalam PODS '17. New York, NY, USA: Association for Computing Machinery, Mei 2017, hlm. 123–135. doi: 10.1145/3034786.3056120.
- [14] “React Native · Learn once, write anywhere.” Diakses: 23 September 2025. [Daring]. Tersedia pada: <https://reactnative.dev/>
 - [15] “Node.js — Run JavaScript Everywhere.” Diakses: 23 September 2025. [Daring]. Tersedia pada: <https://nodejs.org/id>
 - [16] “Express - Node.js web application framework.” Diakses: 23 September 2025. [Daring]. Tersedia pada: <https://expressjs.com/>
 - [17] “Expo Go,” Expo. Diakses: 23 September 2025. [Daring]. Tersedia pada: <https://expo.dev/go>
 - [18] S. R. S. A. Maamari dan M. Nasar, “A Comparative Analysis of NoSQL and SQL Databases: Performance, Consistency, and Suitability for Modern Applications with a Focus on IoT,” *East Journal of Computer Science*, vol. 1, no. 2, hlm. 10–15, Apr 2025, doi: 10.63496/ejcs.Vol1.Iss2.76.
 - [19] creativeans, “UI/UX Design: Pengertian, Prinsip, dan Teknik | Creativeans.” Diakses: 23 September 2025. [Daring]. Tersedia pada: <https://www.creativeans.com/resources/ui-ux-design-pengertian-prinsip-dan-teknik/>
 - [20] E. Grochowski, R. Ronen, J. Shen, dan H. Wang, “Best of both latency and throughput,” dalam *IEEE International Conference on Computer Design: VLSI in Computers and Processors, 2004. ICCD 2004. Proceedings.*, Okt 2004, hlm. 236–243. doi: 10.1109/ICCD.2004.1347928.